AGRUPAMENTO DE ESCOLAS TEMPLÁRIOS

Escola Secundária de Jácome Ratton – Tomar

CURSO PROFISSIONAL

TÉCNICO DE INFORMÁTICA-SISTEMAS

PROVA DE APTIDÃO PROFISSIONAL

Gestor de Palavras-Passe



Pedro Silva

Nº 33882

PROFESSOR(A) ORIENTADOR(A): Ana Rosa

Ano letivo 2021/2022

SELO DE CONFORMIDADE EQAVET · PESSOAS 2030 · PORTUGAL 2030 · Cofinanciado pela União Europeia

ÍNDICE

I. Introdução 1

II. Projeto 2

1. Desenvolvimento 2

1.1. Metodologia 2

1.2. Resultados/produtos 8

1.4. Análise crítica 8

2. Conclusões 8

3. Bibliografia 8

4. Anexos 8

I. Introdução

No âmbito do Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos, este projeto de Prova de Aptidão Profissional (PAP) tem como objetivo o desenvolvimento de uma aplicação para a gestão de palavras-passe. Numa fase posterior, a aplicação permitirá também o armazenamento seguro de ficheiros através de técnicas de encriptação.

A aplicação pretende dar resposta ao problema atual da utilização de palavras-passe fracas e repetidas em diferentes serviços, bem como à necessidade de proteger documentos sensíveis de utilizadores com poucos conhecimentos na área da segurança da informação.

Com o aumento do número de contas online e da quantidade de dados pessoais e profissionais armazenados em formato digital, torna-se essencial disponibilizar ferramentas simples e seguras que auxiliem os utilizadores a guardar credenciais e ficheiros num “cofre” protegido. Assim, este projeto procura conjugar uma interface intuitiva com boas práticas de segurança, garantindo a proteção dos dados armazenados.

II. Projeto

1. Desenvolvimento

1.1. Metodologia

Este projeto foi desenvolvido por etapas. Na primeira etapa, procedi à identificação das necessidades da aplicação, nomeadamente as funcionalidades a implementar e outros requisitos relevantes para o seu correto funcionamento.

Etapa 2 – Aquisição de Conhecimentos

Previamente ao início do desenvolvimento da aplicação, foi necessário proceder à aquisição de conhecimentos fundamentais relativos às tecnologias, ferramentas e conceitos que seriam utilizados ao longo do projeto. Esta etapa revelou-se essencial para garantir uma base sólida de compreensão técnica, permitindo uma implementação mais eficiente das funcionalidades pretendidas.

Base de Dados – MongoDB

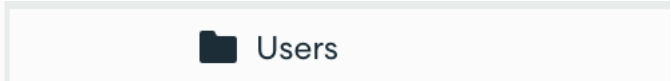
O primeiro domínio de estudo incidiu sobre a tecnologia de base de dados a utilizar. Após uma análise comparativa entre as opções disponíveis, optei pelo MongoDB, uma base de dados não relacional orientada a documentos. Esta escolha fundamentou-se em dois motivos principais.

Em primeiro lugar, o MongoDB destaca-se pela sua versatilidade e flexibilidade no armazenamento de dados. Ao contrário das bases de dados relacionais tradicionais, que exigem uma estrutura rígida e previamente definida, o MongoDB permite uma maior adaptabilidade na organização da informação.

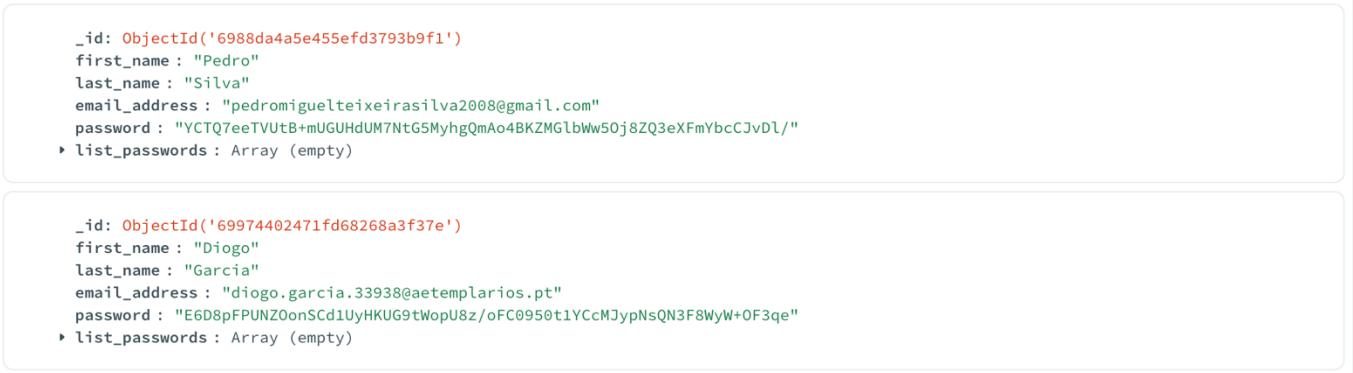
Em segundo lugar, a plataforma MongoDB Atlas possibilita a criação de uma conta gratuita que disponibiliza automaticamente a base de dados na nuvem, funcionando em servidores geridos pela própria MongoDB. Esta característica elimina a necessidade de configuração e manutenção de infraestrutura local, simplificando o processo de desenvolvimento.

No que concerne à estrutura organizacional do MongoDB, este opera através de dois conceitos fundamentais: collections e documents. As collections correspondem, de forma simplificada, às tabelas existentes numa base de dados relacional, enquanto os documents equivalem aos registos dessas tabelas. Contudo, ao contrário do SQL, os documentos são armazenados em formato BSON (Binary JSON), conferindo maior flexibilidade na estrutura dos dados.

Collections



Documents



Linguagem de Programação – C# e XAML

A fase subsequente consistiu na aquisição de conhecimentos relativos à linguagem C#, uma vez que nunca havia utilizado esta linguagem anteriormente. Paralelamente, procedi à aprendizagem do XAML (eXtensible Application Markup Language), utilizado para a construção da interface gráfica.

Criptografia e Segurança

Neste âmbito, foi fundamental compreender a distinção entre encriptação e funções de hash. A encriptação consiste num processo reversível, no qual a informação original pode ser recuperada através de uma chave de desencriptação. Por outro lado, as funções de hash constituem processos unidirecionais, onde a informação original não pode ser recuperada, sendo utilizadas para armazenamento seguro de palavras-passe.

A aquisição destes conhecimentos proporcionou uma base teórica sólida, indispensável para as fases subsequentes do desenvolvimento.

Etapa 3 – Desenvolvimento da Aplicação

A terceira e última etapa do projeto correspondeu à fase de desenvolvimento da aplicação. Esta fase integrou várias subetapas, incluindo a implementação das funcionalidades definidas, a aplicação de mecanismos de segurança e a realização de testes com o objetivo de garantir o correto funcionamento da aplicação.

Segurança da Palavra-passe Mestra

Num gestor de palavras-passe, a segurança da palavra-passe mestra constitui um elemento absolutamente fundamental, uma vez que esta representa a única barreira de proteção entre um potencial atacante e todas as credenciais armazenadas pelo utilizador. Para garantir a máxima segurança deste componente crítico, optei pela utilização da função de hash Argon2, vencedora do Password Hashing Competition em 2015, uma competição internacional que teve como objetivo selecionar o algoritmo mais seguro e robusto para o armazenamento de palavras-passe.

O funcionamento desta função baseia-se na transformação da palavra-passe original através de um cálculo matemático complexo, resultando numa sequência de caracteres completamente irreconhecível e impossível de reverter para obter a palavra-passe original.

Para a geração do hash da palavra-passe mestra, foi necessário definir um conjunto de configurações essenciais que determinam o nível de segurança do processo:

Em primeiro lugar, implementei a utilização de um salt aleatório. O salt consiste numa combinação de caracteres gerada aleatoriamente que é adicionada à palavra-passe do utilizador antes do cálculo do hash. Esta técnica tem como principal objetivo prevenir os denominados ataques de tabela, também conhecidos como rainbow table attacks. Estes ataques consistem na utilização de tabelas pré-calculadas que contêm milhões de combinações de palavras-passe comuns e os respetivos valores de hash. Sem a utilização de um salt, um atacante poderia simplesmente comparar o hash armazenado na base de dados com as entradas da sua tabela pré-calculada, identificando rapidamente a palavra-passe correspondente. Contudo, com a adição de um salt único para cada utilizador, mesmo que dois utilizadores possuam a mesma palavra-passe, os valores de hash resultantes serão completamente diferentes, tornando as tabelas pré-calculadas totalmente ineficazes.

Em segundo lugar, defini o número de iterações do algoritmo. Este parâmetro determina quantas vezes o cálculo do hash é executado de forma encadeada, ou seja, calcula-se o hash da palavra-passe original, posteriormente calcula-se o hash desse resultado, e assim sucessivamente até atingir o limite definido. Implementei um número de iterações consideravelmente elevado, o que aumenta significativamente o tempo necessário para calcular cada hash e, consequentemente, dificulta tentativas de ataques por força bruta.

Em terceiro lugar, configurei a quantidade de memória RAM utilizada durante o processo de cálculo do hash. Esta configuração reveste-se de particular importância na proteção contra atacantes que utilizam hardware especializado. Defini um valor elevado de memória necessária para o cálculo, o que constitui uma medida de segurança extremamente eficaz. A razão para tal prende-se com o facto de que atacantes frequentemente recorrem a placas gráficas (GPUs) ou hardware especializado (ASICs) para realizar ataques de força bruta, uma vez que estes dispositivos conseguem executar milhões de cálculos por segundo. Contudo, este tipo de hardware possui tipicamente quantidades limitadas de memória dedicada. Ao exigir uma quantidade elevada de memória para cada cálculo de hash, torna-se economicamente inviável e tecnicamente impraticável realizar ataques em larga escala com este tipo de equipamento, uma vez que cada tentativa de cálculo requer a alocação de uma quantidade substancial de memória.

Por fim, defini que o processo de cálculo deve utilizar oito núcleos do processador em paralelo. Esta configuração, denominada paralelismo, permite distribuir a carga computacional por múltiplos núcleos, otimizando o desempenho do algoritmo em sistemas legítimos enquanto adiciona uma camada adicional de complexidade para potenciais atacantes.

```
var argon2 = new Argon2id(Encoding.UTF8.GetBytes(password)) //converte a password de texto para bytes
{
    Salt = salt,
    DegreeOfParallelism = 8, // número de threads a serem usadas
    Iterations = 8, // número de iterações
    MemorySize = 131072 // memória usada em KB (128 MB)
};
```

Após a aplicação de todas estas configurações, o hash resultante é armazenado de forma segura na base de dados. Importa salientar que estas mesmas definições são posteriormente utilizadas durante o processo de autenticação: quando o utilizador introduz a sua palavra-passe no ecrã de login, a aplicação calcula o hash da palavra-passe inserida utilizando os mesmos parâmetros, efetua a leitura do hash armazenado na base de dados e compara ambos os valores para verificar se a palavra-passe introduzida corresponde à palavra-passe correta.

Verificação por Correio Eletrónico

Com o objetivo de proporcionar uma camada adicional de segurança para a conta do utilizador, que irá conter dados sensíveis correspondentes a múltiplas palavras-passe, implementei um sistema de verificação por correio eletrónico utilizando a API do Gmail. Este mecanismo funciona de forma semelhante a uma autenticação de dois fatores, sendo ativado tanto no processo de criação de conta como no processo de login.

O funcionamento deste sistema consiste no envio automático de uma mensagem de correio eletrónico para o endereço associado à conta do utilizador, contendo um código de verificação aleatório composto por oito dígitos. Este código possui uma validade temporal limitada a cinco minutos, após os quais expira e torna-se inválido, garantindo assim que códigos antigos não possam ser reutilizados. O utilizador deve então introduzir o código recebido na aplicação, onde é efetuada a verificação da sua correspondência e validade.

Para que o envio de mensagens de correio eletrónico seja executado com sucesso, a aplicação necessita de obter autorização por parte dos serviços da Google. Este processo de autorização é realizado através do protocolo OAuth 2.0, no qual a aplicação recebe tokens de acesso. Estes tokens funcionam de forma análoga a um cartão de autorização digital, certificando que foi concedida permissão à aplicação para executar determinadas ações em nome do utilizador. No contexto deste projeto, os tokens autorizam a aplicação a utilizar uma conta de correio eletrónico específica, criada exclusivamente para este propósito, para o envio das mensagens contendo os códigos de verificação.

```
using (var stream = new FileStream(credentialsPath, FileMode.Open, FileAccess.Read)) //
{
    string tokenPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "token"); //
    credential = await GoogleWebAuthorizationBroker.AuthorizeAsync(
        GoogleClientSecrets.FromStream(stream).Secrets, // Converte o credentials.json
        Scopes, // Permissões que a app está a pedir (neste caso, só enviar emails)
        "user", //
        CancellationToken.None, // Não permito cancelamento da autorização
        new FileDataStore(tokenPath, true) // Guarda o token de acesso na pasta "token"
    );
}
```

1.2. Resultados/produtos

1.3. Análise crítica

(inclui dificuldades/obstáculos e formas de resolução, …)

2. Conclusões

3. Bibliografia

4. Anexos